SCM - Schema Change Manager

A lightweight, extensible database schema management tool written in Python. Supports plain SQL migration files and works with any database that has a Python DB-API 2.0 driver. Inspired by dbmate and golang-migrate.

Features

- Plain SQL migration files -- no proprietary DSL
- Two file styles: single-file (dbmate) or two-file (golang-migrate)
- `init`, `up`, `down`, `status`, `new`, `reset`, `undo`, `learn`, `version`, and `env-from-config` commands
- Timestamp-versioned migrations -- `new` auto-increments the timestamp if a collision exists, so running it multiple times in the same second always produces unique versions
- Tracks applied migrations in a `schema_migrations` table compatible with dbmate (`varchar(128)` version column)
- Built-in drivers for PostgreSQL, MySQL/MariaDB, SQL Server, SQLite, Oracle, SAP HANA, Snowflake, FairCom cTree, and Firebird
- `.env` file support compatible with dbmate
- `env-from-config` pulls connection settings directly from WaSQL's `config.xml`
- Easily extended to support any database with a Python driver

---

Requirements

Python 3.8+. Install the driver package for your database:

Database	Package	Install
PostgreSQL	psycopg2	`pip install psycopg2-binary`
MySQL / MariaDB	mysql-connector-python or pymysql	`pip install mysql-connector-python`
SQL Server	pyodbc	`pip install pyodbc`
SQLite	built-in	nothing
Oracle	oracledb or cx_Oracle	`pip install oracledb`
SAP HANA	hdbcli	`pip install hdbcli`
Snowflake	snowflake-connector-python	`pip install snowflake-connector-python`
FairCom cTree	pyodbc + Faircom ODBC Driver	`pip install pyodbc` + install Faircom driver

Database	Package	Install
Firebird	fdb	`pip install fdb`

MySQL connections try `mysql-connector-python` first and fall back to `pymysql` automatically -- install whichever you have. Both use `charset='utf8mb4'` by default.

Installation

No installation required. The repo includes wrapper scripts so you can call `scm` directly without typing `python scm.py` every time.

Windows -- `scm.bat` is picked up automatically by CMD and PowerShell when run from the project directory (or anywhere on `%PATH%`):

```
 scm status
scm --db hana-t2 up
```

Linux / macOS -- make `scm.sh` executable once, then call it as `scm`:

```
 chmod +x scm.sh

# Optionally symlink onto your PATH:
ln -s "$(pwd)/scm.sh" /usr/local/bin/scm
```

Both scripts simply delegate to `python3 scm.py "$@"` -- all flags and arguments pass through unchanged.

Quick Start

The fastest way to get started is with `env-from-config` (WaSQL projects) or `init`:

```
 # WaSQL project: pull settings from config.xml, create migrations folder and .env in one step
scm env-from-config wasql_test_17

# Non-WaSQL project: create migrations folder and .env stub
scm init

# Edit .env to set DATABASE_URL, then:
scm new create_users_table
scm up
```

.env File

`scm` automatically loads `.env` from the current directory before running. All settings can be placed here so you never have to pass flags on every command.

```
# .env -- minimum required
DATABASE_URL=mysql://user:pass@localhost/mydb

# Optional settings (shown with defaults)
MIGRATION_STYLE=one
# MIGRATIONS_DIR=./migrations
# MIGRATIONS_TABLE=schema_migrations
```

Supported syntax

```
# Plain
DATABASE_URL=postgres://user:pass@host/mydb

# Quoted
DATABASE_URL="postgres://user:pass@host/mydb"

# export prefix (optional)
export DATABASE_URL="postgres://user:pass@host/mydb"

# Inline comments
DATABASE_URL=postgres://user:pass@host/mydb  # production db
```

Existing environment variables always take precedence over `.env`. Override for one command by exporting in the shell:

```
DATABASE_URL="postgres://user:pass@staging-host/mydb" scm status
```

To use a different `.env` file:

```
scm --env-file .env.production status
scm --env-file .env.staging up
```

.env variables reference

Variable	Aliases	Default	Description
`DATABASE_URL`	--	*(required)*	Database connection URL
`MIGRATION_STYLE`	--	`one`	File style for `new`: `one` / `dbmate` = single file, `two` / `golang-migrate` = separate up/down files
`MIGRATIONS_DIR`	`DBMATE_MIGRATIONS_DIR`	`./migrations`	Path to migrations directory
`MIGRATIONS_TABLE`	`DBMATE_MIGRATIONS_TABLE`	`schema_migrations`	Name of the tracking table in the

Variable	Aliases	Default	Description
			database

Connection URL

The database connection URL is resolved in this order (first match wins):

1. `--url` flag
2. `DATABASE_URL` environment variable
3. `DATABASE_URL` in `.env`

```
 # PostgreSQL
DATABASE_URL="postgres://user:pass@localhost:5432/mydb"
DATABASE_URL="postgres://user:pass@db-host:5432/mydb?sslmode=require"

# MySQL / MariaDB (mysqli is also accepted)
DATABASE_URL="mysql://user:pass@localhost:3306/mydb"

# SQL Server
DATABASE_URL="mssql://user:pass@localhost:1433/mydb"

# SQLite
DATABASE_URL="sqlite:///path/to/db.sqlite3"

# Oracle (path is the service name)
DATABASE_URL="oracle://user:pass@host:1521/myservice"

# SAP HANA
DATABASE_URL="hana://user:pass@host:39015/"

# Snowflake (warehouse, schema, role are optional query params)
DATABASE_URL="snowflake://user:pass@myaccount.snowflakecomputing.com/mydb?warehouse=COMPUTE_W

# FairCom cTree (requires Faircom ODBC Driver installed)
DATABASE_URL="ctree://user:pass@host:6597/mydb"

# Firebird
DATABASE_URL="firebird://user:pass@host/path/to/database.fdb"
```

The `--url` flag overrides everything for a single command:

```
scm --url postgres://user:pass@host/mydb status
```

Commands

`init` -- Set up a new project

```
 scm init
scm --path ./db/migrations init
```

Creates the migrations directory and a `.env` stub if they don't already exist. Existing files are left untouched. Shows next steps when done.

```
   created migrations
  created .env

Next steps:
  Edit .env and set DATABASE_URL, or run:
    scm env-from-config <name>
  scm new <migration_name>
  scm up
```

The generated `.env` stub:

```
 DATABASE_URL=
MIGRATION_STYLE=one
# MIGRATIONS_DIR=./migrations
# MIGRATIONS_TABLE=schema_migrations
```

`env-from-config` -- Configure from WaSQL config.xml

```
 # List all supported databases defined in config.xml
scm env-from-config

# Write DATABASE_URL to .env and run init
scm env-from-config <name>
```

Reads `../config.xml` (relative to `scm.py`) and builds a `DATABASE_URL` from the named `<database>` entry's attributes (`dbtype`, `dbhost`, `dbport`, `dbuser`, `dbpass`, `dbname`). Creates or updates `DATABASE_URL` in `.env` in-place, then automatically runs `init` to create the migrations folder.

Only database entries with supported dbtypes are listed: `postgres`, `mysql`, `mysqli`, `mariadb`, `mssql`, `sqlserver`, `sqlite`, `ctree`, `firebird`, `hana`, `snowflake`, `oracle`.

Snowflake entries automatically encode `dbschema`, `dbwarehouse`, and `dbrole` as query parameters in the URL. Passwords with special characters are percent-encoded.

```
 scm env-from-config wasql_test_17
# Created .env.wasql_test_17
#   DATABASE_URL=mysql://wasql_dbuser:***@localhost/wasql_test_17
#   created migrations/wasql_test_17
#   created .env.wasql_test_17
```

Use `--config` to point at a different `config.xml`:

```
 scm --config /path/to/config.xml env-from-config mydb
```

`new` -- Create a migration

```
scm new <name> [--style one|dbmate|two|golang-migrate]
```

Generates a timestamped migration file (or pair) in the migrations directory. The style defaults to `MIGRATION_STYLE` in `.env` (which defaults to `one`).

```
# Single-file style (default when MIGRATION_STYLE=one or MIGRATION_STYLE=dbmate)
scm new create users table
# → migrations/20240601120000_create_users_table.sql

# Two-file style (use --style two or set MIGRATION_STYLE=two in .env)
scm new create users table --style two
# → migrations/20240601120000_create_users_table.up.sql
# → migrations/20240601120000_create_users_table.down.sql

# Custom migrations path
scm --path ./db/migrations new create_orders
```

Migration names must contain only letters, digits, underscores, and hyphens, and must start with a letter or digit.

Timestamp collision avoidance -- if the current second is already taken by an existing file, `new` increments by one second until it finds a free slot. Running `new` five times rapidly will produce `...12`, `...13`, `...14`, `...15`, `...16` with no manual intervention needed.

`status` -- Show migration status

```
scm status
```

Lists all migration files and whether each has been applied. Also reports versions recorded in the database that have no file on disk (orphaned).

When writing to a terminal, output is color-coded:

Color	Meaning
Gray / dim	Applied -- already in the database
Green	Pending -- not yet applied
Yellow	Orphaned -- applied in DB but file is missing

```
Version          Label                                Status      Down?
-----------------------------------------------------------------------
20240601120000   create users table                   applied     yes   ← gray
20240602083000   add email index                      applied     yes   ← gray
20240603094500   add orders table                     pending     yes   ← green
20240604110000   add audit log                        pending     no    ← green
20240530000000   <file missing>                       orphaned    ?     ← yellow

4 migrations: 2 applied, 2 pending,
1 orphaned (applied in DB but no file on disk).
```

Colors are suppressed automatically when output is piped or redirected.

The `Down?` column indicates whether a rollback migration exists. Orphaned entries mean a version was applied but the file was later deleted -- investigate before running `up` or `down`.

`up` -- Apply migrations

```
# Apply all pending migrations
scm up

# Apply the next N pending migrations
scm up 1
scm up 3
```

Each migration runs in a transaction. On failure the transaction rolls back and the command exits immediately -- migrations applied before the failure are preserved.

SQL statements are split by a context-aware parser that handles semicolons inside `-- line comments`, `/* block comments */`, and `'string literals'`. PostgreSQL migrations are passed as a single string so dollar-quoted blocks (`$$...$$`) work natively. SQL Server migrations are split on `GO` batch separators in addition to semicolons.

`down` -- Roll back migrations

```
# Roll back the last migration (default)
scm down

# Roll back the last N migrations
scm down 3
```

If a migration has no down script, `down` exits with an error rather than silently skipping.

`reset` -- Wipe migration history and files

```
# Interactive -- prompts for confirmation
scm reset

# Skip confirmation prompt
scm reset --force

# Target a specific migrations directory
scm --path ./db/migrations reset --force
```

Deletes all rows from the `schema_migrations` tracking table **and** removes every `.sql` file from the migrations directory. Use this to wipe a dev environment and start from scratch.

> **Warning:** This is destructive and irreversible. The database schema itself is **not** touched -- only the migration history records and migration files are removed.

```
# Typical dev reset workflow
scm reset --force
# (re-create your migration files)
scm up
```

`undo` -- Delete pending migration files

```
scm undo
```

Lists all pending (unapplied) migrations numbered, prompts you to select which to remove, then deletes the file(s) from disk and removes any tracking record from the database — as if the migration was never created.

```
Pending migrations:
  1. 20240603094500 add orders table
  2. 20240604110000_add_audit_log

Enter number(s) to undo (e.g. 1  or  1,3  or  1-3), blank to cancel: 2
  Deleted  20240604110000_add_audit_log.sql
```

Selection supports single numbers (`1`), comma or space separated lists (`1,3`), and ranges (`1-3`). Blank input cancels with no changes made.

> **Note:** This removes migration files that have not yet been applied. To roll back an already-applied migration, use `scm down` instead.

`learn` -- Quick-start reference

```
scm learn
```

Prints a formatted quick-start reference covering setup, daily workflow, migration file format, tips, and global flags. No database connection required. Output is color-coded when writing to a terminal and plain text when piped or redirected.

`version` -- Print version

```
scm version
# scm 1.0.0
```

Requires no database connection.

Global Flags

Flag	Default	Description
`--url URL`	--	Database URL. Overrides `DATABASE_URL` and `.env`
`--db NAME`	--	Database name. Derives `--env-file` and `--path` automatically (see below)
`--env-file FILE`	`.env` (or `.env.<db>`)	Path to `.env` file
`--path DIR`	`./migrations` (or `./migrations/<db>`)	Migrations directory
`--config FILE`	`../config.xml`	Path to WaSQL `config.xml` (used by `env-from-config`)

File Naming

Migration files are sorted and applied by their numeric version prefix. Timestamps (recommended) or plain integers both work.

```
20240601120000_create_users.up.sql
20240601120000_create_users.down.sql
20240602083000_add_email_index.up.sql
```

Rules:

- Version prefix must be all digits
- Separator is `_` or `-`
- Two-file style: suffixes are `.up.sql` and `.down.sql`
- Single-file style: suffix is `.sql` with `-- migrate:up` / `-- migrate:down` markers
- If both styles exist for the same version, two-file takes precedence
- Duplicate version numbers are an error and reported with both filenames

---

File Styles

Single-file style (dbmate) -- default

`20240601120000_create_users.sql`

```
-- migrate:up
CREATE TABLE users (
    id         SERIAL PRIMARY KEY,
    name       VARCHAR(255) NOT NULL,
    email      VARCHAR(255) NOT NULL UNIQUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
-- migrate:down
DROP TABLE users;
```

Two-file style (golang-migrate)

`20240601120000_create_users.up.sql`

```
CREATE TABLE users (
    id         SERIAL PRIMARY KEY,
    name       VARCHAR(255) NOT NULL,
    email      VARCHAR(255) NOT NULL UNIQUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

`20240601120000_create_users.down.sql`

```
DROP TABLE users;
```

Both styles can coexist in the same migrations directory across different versions.

Schema Migrations Table

`scm` creates and manages the tracking table automatically. The table name defaults to `schema_migrations` and can be changed via `MIGRATIONS_TABLE` (or `DBMATE_MIGRATIONS_TABLE`) in `.env`.

```
-- PostgreSQL
CREATE TABLE schema_migrations (
    version    varchar(128) PRIMARY KEY NOT NULL,
    applied_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- MySQL / MariaDB
CREATE TABLE schema_migrations (
    version    varchar(128) PRIMARY KEY NOT NULL,
    applied_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- SQL Server
CREATE TABLE schema_migrations (
    version    varchar(128) NOT NULL PRIMARY KEY,
    applied_at DATETIME2 NOT NULL DEFAULT GETUTCDATE()
);
```

The version stored is the numeric prefix of the migration filename as a string (e.g. `'20240601120000'`). Using `varchar(128)` matches the schema created by dbmate, so a `schema_migrations` table created by either tool is fully interoperable with the other. If a table already exists with a different column type (e.g. `BIGINT`), scm reads it correctly by converting values to integers for comparison internally.

Per-Database Migrations (onemcp / WaSQL)

When working with multiple databases via the onemcp MCP server, use `--db <name>` to scope all scm commands to a specific database. The flag automatically resolves:

- `--env-file` → `.env.<name>` (e.g. `.env.hana-t2`)
- `--path` → `./migrations/<name>` (e.g. `./migrations/hana-t2`)

Setup (one time per database)

```
# Pulls DATABASE URL from config.xml, writes .env.hana-t2, creates migrations/hana-t2/
scm env-from-config hana-t2
```

Daily use

```
# After "use hana-t2" in the MCP session:
scm --db hana-t2 status
scm --db hana-t2 new add_orders_table
scm --db hana-t2 up
```

Resulting layout

```
.env.hana-t1
.env.hana-t2
.env.ods
.env.dexpdq
migrations/
  hana-t1/
  hana-t2/
  ods/
  dexpdq/
```

You can still override either default explicitly:

```
# Use --db for the database but a custom migrations path
scm --db hana-t2 --path ./db/custom-migrations status
```

Multiple Repos / Multiple Databases

Each repo keeps its own `.env` with its own `DATABASE_URL`. Running `scm` from inside a repo picks up that repo's `.env` automatically.

```
repo-a/
  .env                # DATABASE_URL=postgres://...prod-a.../app_a
  migrations/

repo-b/
  .env                # DATABASE_URL=mysql://...prod-b.../app_b
  migrations/
```

Makefile pattern

```
MIGRATE = scm --path ./migrations

.PHONY: migrate-new migrate-up migrate-down migrate-status

migrate-new:
  $(MIGRATE) new $(name)

migrate-up:
  $(MIGRATE) up

migrate-down:
  $(MIGRATE) down $(n)

migrate-status:
  $(MIGRATE) status
```

Adding a Custom Driver

Subclass `BaseDriver`, implement five methods, and register with `@register_driver`. Use `self.table` (not a hardcoded string) for the tracking table name so `MIGRATIONS_TABLE` in `.env` is respected.

```
@register_driver(['mydb'])
class MyDBDriver(BaseDriver):

    def connect(self):
        import mydb_driver
        p = urlparse(self.url)
        self.conn = mydb_driver.connect(
            host=p.hostname,
            port=p.port or 5000,
            user=p.username,
            password=p.password,
            database=p.path.lstrip('/'),
        )

    def ensure_migrations_table(self):
        try:
            self.execute(f"""
                CREATE TABLE {self.table} (
                    version    varchar(128) NOT NULL PRIMARY KEY,
                    applied_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
                )
            """)
            self.commit()
        except Exception:
            self.rollback()  # table already exists

    def applied_versions(self):
        cur = self.execute(f"SELECT version FROM {self.table} ORDER BY version")
        return {{int(row[0]) for row in cur.fetchall()}}

    def record_migration(self, version):
        self.execute(f"INSERT INTO {self.table} (version) VALUES (?)", [str(version)])
```

```
    def remove_migration(self, version):
        self.execute(f"DELETE FROM {self.table} WHERE version = ?", [str(version)])
```

The driver is activated automatically when its URL scheme is detected in `DATABASE_URL`.

Comparison with dbmate and golang-migrate

Feature	scm	dbmate	golang-migrate
Language	Python	Go	Go
PostgreSQL	yes	yes	yes
MySQL / MariaDB	yes	yes	yes
SQL Server (+ GO batches)	yes	no	yes
SQLite	yes	yes	yes
Oracle	yes (built-in)	no	no
SAP HANA	yes (built-in)	no	no
Snowflake	yes (built-in)	no	no
FairCom cTree	yes (built-in)	no	no
Firebird	yes (built-in)	no	no
Single-file migrations	yes	yes	no
Two-file migrations	yes	no	yes
Down migrations	yes	yes	yes
`.env` support	yes	yes	no
`new` command	yes	yes	no
`init` command	yes	yes	no
Configurable tracking table	yes	yes	yes
WaSQL config.xml integration	yes	no	no
`goto` version	no	no	yes
Extensible drivers	yes	no	limited
Dependencies	pip packages only	none (binary)	none (binary)

Writing Safe Migrations

Always required

- **Write a down migration.** Every `up` should have a matching `down` unless the operation is truly irreversible. If skipping, add a comment explaining why.
- **Use `IF EXISTS` / `IF NOT EXISTS` guards** so re-runs after partial failure don't error: `sql CREATE TABLE IF NOT EXISTS orders (...); DROP TABLE IF EXISTS old_sessions; DROP INDEX IF EXISTS idx_old;`
- **Never modify an applied migration.** Once run anywhere, treat it as immutable. Create a new migration to correct it.
- **Never include secrets, passwords, or PII** in migration SQL.

Adding columns to existing tables

Adding a `NOT NULL` column without a default fails on tables that already have rows:

```
 -- Safe: add nullable first, backfill, then constrain in a later migration
ALTER TABLE users ADD COLUMN tier VARCHAR(20);

-- Also safe: default that applies retroactively
ALTER TABLE users ADD COLUMN tier VARCHAR(20) NOT NULL DEFAULT 'free';
```

Destructive operations

Operation	Risk	Mitigation
`DROP TABLE`	Permanent data loss	Always `IF EXISTS`; ensure down recreates it
`DROP COLUMN`	Permanent data loss	Deploy app changes first
`TRUNCATE`	Empties entire table	Rarely appropriate in a migration
`DELETE` without `WHERE`	Empties entire table	Always add a `WHERE` clause
Type change	May truncate data	Test on a production-volume copy first
Rename table/column	Breaks running app	Must coordinate with a code deploy

Database-specific DDL behaviour

PostgreSQL -- DDL is transactional. `CREATE INDEX CONCURRENTLY` cannot run inside a transaction; put it alone in its own migration file.

MySQL / MariaDB -- DDL is **not transactional**. A failed migration may partially apply and cannot be automatically rolled back. Keep each DDL change in its own file.

SQL Server -- DDL is transactional. Use `GO` to separate batches that must run independently.

SQLite -- DDL is transactional. Does not support `ADD COLUMN IF NOT EXISTS` or `DROP COLUMN` before version 3.35.0.

Data migrations

- Keep data migrations separate from schema migrations.
- Add a `WHERE` clause to every `UPDATE` and `DELETE`.
- For large tables, batch updates in chunks to avoid long locks.

Security Considerations

Credentials

- Store `DATABASE_URL` in `.env`, never hardcoded in scripts or committed to version control.
- Add `.env` to `.gitignore`.
- Use a dedicated migration user with only the permissions needed -- not the application's runtime user or a superuser.
- The `--url` flag exposes credentials in shell history and process lists. Prefer `.env`.

Principle of least privilege

```
 -- PostgreSQL: migration user
GRANT CONNECT ON DATABASE mydb TO migrator;
GRANT USAGE, CREATE ON SCHEMA public TO migrator;
GRANT INSERT, DELETE ON schema_migrations TO migrator;

-- MySQL: migration user
GRANT CREATE, ALTER, DROP, INDEX, REFERENCES ON mydb.* TO 'migrator'@'%';
GRANT INSERT, DELETE ON mydb.schema_migrations TO 'migrator'@'%';
```

Concurrency

`scm` does not acquire an advisory lock. Running two `up` commands simultaneously against the same database will cause the second to fail on the duplicate-key insert -- but on MySQL any DDL already executed is permanent. Ensure only one migration process runs at a time (CI serialization, deployment locks, etc.).

Running in CI/CD

- Mask `DATABASE_URL` in CI settings so it never appears in logs.
- Run `status` before `up` so the log shows exactly what will be applied.
- Consider `up 1` in automated pipelines so a bad migration fails fast.

Troubleshooting

`Migrations directory not found` Run `scm init` (or `env-from-config`) to create it.

`No -- migrate:up marker found` Single-file migrations must contain a `-- migrate:up` line. Check for typos.

`No down migration for X -- cannot roll back` The migration has no down script. Add one or roll back manually.

`Unsupported database scheme` The URL scheme is not recognized. Check your `DATABASE_URL` prefix.

`Duplicate migration version N: file1 and file2` Two files share the same numeric prefix. Rename one -- or use `scm new` which avoids this automatically by incrementing the timestamp until a free slot is found.

`Invalid migration name` Names must use only letters, digits, underscores, and hyphens, starting with a letter or digit.

`Failed to connect (...): ...` Connection failed. The password is masked in the error. Check host, port, credentials, and network reachability.

`No MySQL driver found` Install either: `pip install mysql-connector-python` or `pip install pymysql`.

`Database 'X' not found in config.xml` Run `scm env-from-config` (no name) to list supported entries.

`status` shows all migrations as `pending` and applied versions as `orphaned` The `schema_migrations.version` column is a type that doesn't compare equal to the integer versions parsed from filenames (e.g. the table was created by dbmate with a `varchar` column but an older scm stored integers, or vice versa). scm handles this automatically by converting all versions to integers for comparison -- but if you hit this, run `scm reset --force` and re-apply.

Migration applied but schema change is missing The migration may have partially succeeded. Check the database manually. If the tracking row exists but the schema change did not apply, remove the row and re-run `up` after fixing the SQL.